PeakSoft Cyber-Guardian

Rapport Technique — Phases 1 & 2

Plateforme de Securite Industrielle

avec Digital Twin, IA et Supervision Temps Reel

Meriem — Security Intelligence (Track A)

Phase 1 & 2

Rapport Technique de Stage

Plateforme de Sécurité Industrielle PeakSoft Cyber-Guardian

Stagiaire : Meriem

Track : Security Intelligence

Phases : 1 & 2 — Core Fondation, Ingestion Sécurisée, IA & Dashboard React

---

Partie I : Introduction

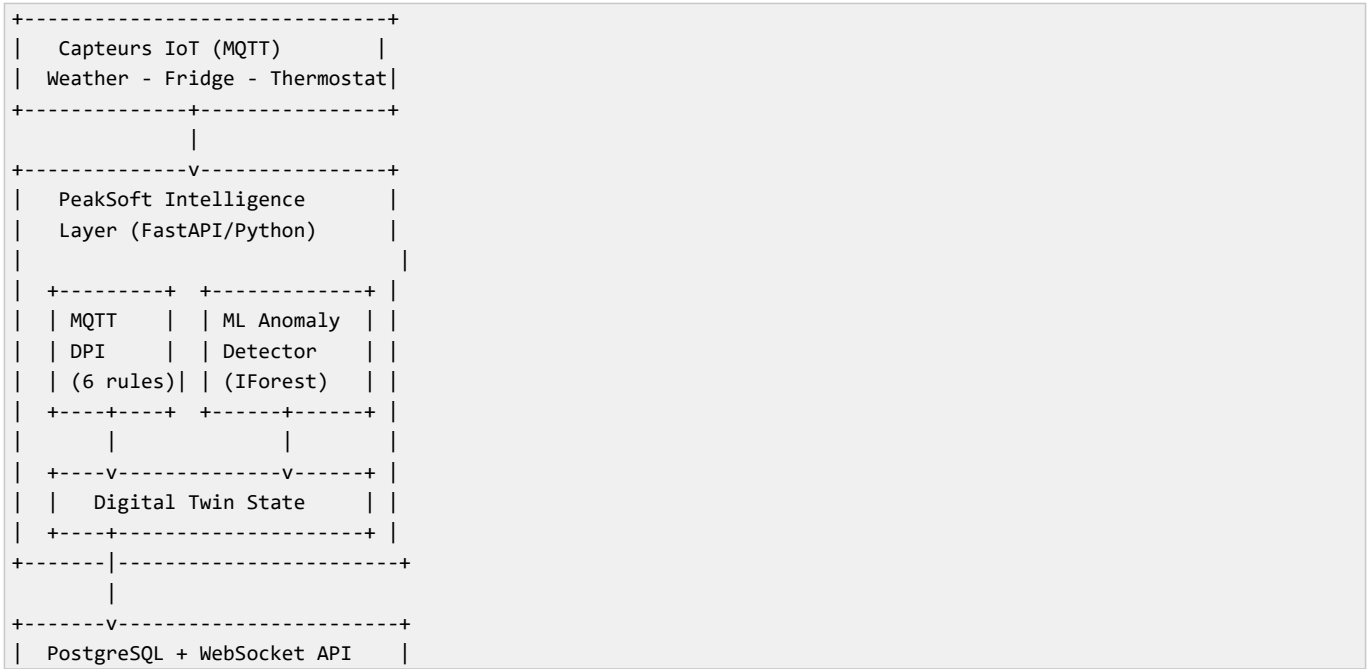
1.1 Contexte Industriel

L'Industrie 4.0 connecte des capteurs IoT (température, humidité) à des réseaux IP via le protocole léger MQTT. Cette ouverture expose les infrastructures critiques à des cyberattaques (DDoS, MITM, Ransomware, Injection de données). Les solutions de sécurité traditionnelles sont insuffisantes face à des attaques qui exploitent les protocoles IoT eux-mêmes.

Le projet PeakSoft Cyber-Guardian répond à ce besoin par une plateforme de cybersécurité qui combine :

- 3 capteurs IoT : Weather (station météo), Fridge (chambre froide), Thermostat (climatisation)
- Digital Twin : Réplique virtuelle synchronisée des équipements physiques
- Deep Packet Inspection (DPI) : Analyse en temps réel du trafic MQTT
- Machine Learning : Détection d'anomalies comportementales par Isolation Forest
- XAI (eXplainable AI) : Explicabilité des décisions via SHAP
- Dashboard React : Interface de supervision temps réel

1.2 Architecture Globale



```
|  + React Frontend (Vite)       |
+--------------------------------+
```

```
---

# Partie II : Phase 1 — Core Fondation & Ingestion Sécurisée

## 2.1 Architecture Base de Données

### Problème
Les applications web écrivent souvent des modèles Pydantic pour l'API ET des modèles SQLAlchemy pour la base de données, ce qui crée une dérive de synchronisation ("Sync Drift").

### Solution : SQLModel + Domain-Driven Design

L'architecture suit le **Domain-Driven Design (DDD)** avec un **Repository Pattern** :
```

```
app/
+-- domains/
    +-- devices/
    |   +-- models.py        # SQLModel (Device, Telemetry)
    |   +-- schemas.py       # Pydantic (validation API)
    |   +-- repository.py    # Acces aux donnees
    |   +-- service.py       # Logique metier
    +-- intelligence/
    |   +-- ml_service.py    # Service ML (Isolation Forest + SHAP)
    |   +-- mqtt_dpi_service.py  # Regles DPI
    |   +-- service.py       # Service OpenSearch
    +-- ml_models/
        +-- models.py        # Tables MLModel, ModelMetric
        +-- schemas.py
        +-- repository.py
        +-- service.py
```

```
**Choix techniques :**
- **SQLModel** : Modele unifie (Pydantic + SQLAlchemy) — elimine la duplication
- **asyncpg** : Driver PostgreSQL asynchrone — pas de blocage du event loop
- **Repository Pattern** : Decouplage entre la logique metier et le stockage
- **Dependency Injection** : FastAPI `Depends` pour injecter services/repositories

### Schema de donnees principal
```

```
Device {
    id: str (PK)
    name: str
    device_type: str (weather, fridge, thermostat)
    digital_twin_state: JSON (derniere telemetrie)
    last_seen: DateTime
    risk_level: str
}

Telemetry {
    id: int (PK)
    device_id: str (FK -> Device.id)
    timestamp: DateTime
    temperature: float
    humidity: float
    pressure: float
    status: str
```

```
}
```

```
### Fichiers cles
- `iot-security-backend/app/domains/devices/models.py`
- `iot-security-backend/app/db/session.py`
- `iot-security-backend/app/db/base_repository.py`

---

## 2.2 MQTT Deep Packet Inspection (DPI)

### Probleme
Les messages MQTT transportent des donnees IoT non filtrees. Un attaquant peut injecter de fausses donnees,
saturer le reseau, ou alterer les mesures.

### Solution : 6 regles DPI

Le service `MqttDPIService` analyse chaque message MQTT recu et applique 6 regles numerotees :

| Regle | Code | Declencheur | Action |
|-------|------|------------|--------|
| DPI-001 | `rule_attack_label` | Label "attack" dans le payload | Alerte critique (level 3) |
| DPI-002 | `rule_noise_factor` | Facteur de bruit > 0.04 | Alerte haute (level 2) |
| DPI-003 | `rule_packet_loss` | Perte de paquet simulee | Alerte haute (level 2) |
| DPI-004 | `rule_out_of_range` | Valeur hors seuils capteur | Alerte variable (level 1-3) |
| DPI-005 | `rule_generic_spike` | Valeur > 1000 | Alerte haute (level 2) |
| DPI-006 | `rule_invalid_humidity` | Humidite < 0% ou > 100% | Alerte basse (level 1) |

**Seuils par type de capteur (DPI-004) :**
```

```
SENSOR_THRESHOLDS = {
    "weather":    {"temperature": (0, 60), "humidity": (0, 100), "pressure": (800, 1200)},
    "fridge":     {"Fridge_Temperature": (-5, 20)},
    "thermostat": {"current_temperature": (10, 50)},
}
```

```
**Architecture :** Singleton expose via `dpi_service`, integre dans `mqtt_service.py`. Chaque message est inspecte
avant persistance.

### Fichiers cles
- `iot-security-backend/app/domains/intelligence/mqtt_dpi_service.py`
- `iot-security-backend/app/services/mqtt_service.py`

---

## 2.3 Simulateur de Streaming Temps Reel

### Probleme
Le dataset TON_IoT est un ensemble de fichiers CSV statiques. Pour tester le DPI et le ML en conditions reelles,
il faut un flux continu de donnees.

### Solution : Simulateur Asynchrone Multi-Capteurs

`mqtt_simulator_ton_iot.py` implemente :
- **Architecture asyncio** : Une coroutine par device, toutes en parallele via `asyncio.gather`
- **3 modes** : `normal` (donnees saines), `attack` (donnees d'attaque), `mixed` (melange aleatoire)
- **Publication MQTT** sur les topics `factory/{device_id}/telemetry`
- **Mapping CSV reel** : Les fichiers du dataset TON_IoT sont lus et streamés avec un delai configurable

**CLI :**
```

```
python scripts/mqtt_simulator_ton_iot.py \
    --host localhost --port 1883 \
    --mode mixed --delay-min 0.5 --delay-max 2.0
```

```
### Fichier cle
- `iot-security-backend/scripts/mqtt_simulator_ton_iot.py`

---

## 2.4 Exploration & Pretraitement des Donnees (EDA)

### Objectif
Analyser 13 fichiers CSV repartis sur 3 capteurs (Weather, Fridge, Thermostat) pour comprendre la structure des
donnees et preparer le Machine Learning.

### Exploration
- **Weather** : 399 189 lignes, temperature 20-50°C, pression negative aberrante -> exclusion
- **Fridge** : 383 369 lignes, cycle high/low (56%/44%), temperature 1-14°C
- **Thermostat** : 291 183 lignes, temperature stable 25-35°C, ON 89% du temps
- **Correlations** : Weather vs Fridge r = -0.94, Weather vs Thermostat r = -0.97

### Feature Engineering

| Feature | Formule | Role |
|---------|---------|------|
| Δt (diff) | X(t) - X(t-1) | Capture les changements brusques d'attaque |
| Moyenne mobile | moyenne(X(t-9)...X(t)) | Lisse le bruit des capteurs |
| Ecart-type mobile | σ(X(t-9)...X(t)) | Mesure la volatilite recente |
| Heure sin/cos | sin(2π×h/24), cos(2π×h/24) | Encodage cyclique du temps |
| Encodage etat | high->1, low->0 | Variables categorieles -> numeriques |
| Z-score (StandardScaler) | (x - μ) / σ | Normalisation des features |

### Fichiers cles
- `notebooks/1_EDA_Exploration_Capteurs_IoT_executed.ipynb`
- `notebooks/2_Preprocessing_FeatureEngineering_ML.ipynb`
- `scripts/phase2_preprocess.py`

---

# Partie III : Phase 2 — Intelligence Artificielle & Dashboard React

## 3.1 Ingénierie MQTT

### Broker Mosquitto
- Deploiement Docker du broker Eclipse Mosquitto
- Configuration avec persistance des messages
- RBAC prepare via fichier de mots de passe monte

### Client MQTT Asynchrone (gmqtt)
Le `MqttService` est integre au cycle de vie FastAPI :
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    asyncio.create_task(mqtt_service.start())  # Background non-bloquant
    yield
    await mqtt_service.stop()  # Arret propre
```

```
Caracteristiques :
- Abonnement au pattern `factory/+/telemetry` (decouverte dynamique)
- Parsing des topics pour extraire le `device_id`
```

```
- Normalisation des timestamps en UTC
- Ecriture asynchrone en base (non-bloquant)
- Validation des cles etrangeres (device_id doit exister)

### Pipeline de Message
```

```
Message MQTT brut
    -> Parsing JSON (try/except)
    -> Extraction device_id du topic
    -> Normalisation timestamp (UTC)
    -> Inspection DPI (6 regles)
    -> Persistance asynchrone (PostgreSQL)
    -> Mise a jour Digital Twin (digital_twin_state, last_seen)
    -> Push WebSocket vers le dashboard React
```

```
### Fichiers cles
- `iot-security-backend/app/services/mqtt_service.py`
- `iot-security-backend/app/main.py` (lifespan)

---

## 3.2 Digital Twin Synchronisation Temps Reel

### Probleme
Le dashboard affiche l'etat des devices, mais sans synchronisation temps reel, les donnees sont perimees des
l'affichage.

### Solution
1. **Mise a jour transactionnelle** : A chaque message MQTT, le `Device.digital_twin_state` (JSON) et
`Device.last_seen` sont mis a jour dans la meme transaction que l'insertion de telemetrie
2. **API d'historique** : `GET /devices/{id}/telemetry` -> dernieres 50 mesures
3. **Integrite temporelle** : `last_seen` n'est mis a jour que par MQTT, pas par les requetes API
4. **WebSocket temps reel** : Endpoint `/ws/stream` pour le push vers le frontend React

---

## 3.3 Dashboard React

### Technologies
- **React 19** + **TypeScript** + **Vite** pour le framework
- **Tailwind CSS** pour le styling
- **Recharts** pour les graphiques
- **Framer Motion** pour les animations
- **WebSocket** pour les donnees temps reel

### Composants principaux

| Composant | Role |
|-----------|------|
| `DashboardPage.tsx` | Vue principale avec Digital Twin et simulateur d'attaques |
| `KpiGrid.tsx` | Cartes KPI (temperature, humidite, score ML) |
| `TimelineChart.tsx` | Graphiques d'evolution temporelle |
| `XaiPanel.tsx` | Panneau d'explicabilite SHAP |
| `AttackSimulator.tsx` | Simulation d'attaques (DDoS, MITM, Ransomware) |
| `TwinStatus.tsx` | Etat du Digital Twin |
| `AlertTable.tsx` | Tableau des alertes DPI |
| `AnimatedCounter.tsx` | Compteurs animes |
| `ThemeContext.tsx` | Themes dark/cyberpunk/matrix/light |

### Flux de donnees
```

```
Capteurs IoT (MQTT)
```

```
    -> Backend FastAPI
        -> Inspection DPI
        -> ML Inference (Isolation Forest + SHAP)
        -> Persistance PostgreSQL
        -> WebSocket push
    -> Frontend React
        -> Mise a jour temps reel des graphiques
        -> Alertes visuelles
        -> Panneau SHAP
```

```
### Fichiers cles
- `frontend/src/pages/DashboardPage.tsx`
- `frontend/src/pages/DevicesPage.tsx`
- `frontend/src/components/XaiPanel.tsx`
- `frontend/src/components/AttackSimulator.tsx`
- `frontend/src/hooks/useTelemetry.ts`

---

## 3.4 Detection d'Anomalies par Isolation Forest

### Principe
L'Isolation Forest isole les anomalies par des coupes aleatoires :
- Un point **normal** necessite beaucoup de coupes pour etre isole (il est entoure)
- Une **anomalie** est isolee rapidement (elle est seule)

### Modeles Entraines

| Modele | Features | Echantillons | Performance moyenne |
|--------|----------|-------------|-------------------|
| Weather | 8 (temperature, humidite, Δt, rolling, heure) | 399 188 | **99.4%** |
| Fridge | 6 (temperature, etat, Δt, rolling, heure) | 383 368 | **98.5%** |
| Thermostat | 5 (temperature, statut, Δt, heure) | 291 182 | **33.2%** (faible) |
| Combine | ~19 (fusion des 3) | 370 972 | **100%** |

### Resultats par type d'attaque

| Attaque | Weather | Fridge | Combine |
|---------|---------|--------|---------|
| DDoS | 100.0% | 97.8% | 100.0% |
| DoS | 100.0% | 99.8% | 100.0% |
| MITM | 100.0% | 100.0% | 100.0% |
| XSS | 100.0% | 99.6% | 100.0% |
| Backdoor | 100.0% | 98.6% | -- |
| Injection | 99.4% | 99.8% | 100.0% |
| Password | 100.0% | 98.5% | 100.0% |
| Ransomware | 97.5% | 93.8% | 100.0% |
| Scanning | 74.2% | 99.4% | 100.0% |

### Analyse
- **Weather (excellent)** : 100% sur 7/9 attaques -- grande variance naturelle (20-50°C) donne un signal fort
- **Fridge (tres bon)** : >93% sur toutes -- la variable `temp_condition` (high/low) est discriminante
- **Thermostat (faible)** : 5-54% -- temperature trop stable (σ=2.4), ON 89% du temps
- **Combine (parfait)** : 100% quand la fusion inter-capteurs est possible

### Pipeline d'inference temps reel
```

```
Nouvelle telemetrie
    -> Feature engineering (Δt, rolling, heure)
    -> Normalisation (StandardScaler)
    -> Score Isolation Forest (score_samples)
    -> SHAP explainability (TreeExplainer)
```

```
    -> Decision : anomalie si score < seuil
    -> Mise a jour du statut Digital Twin
    -> Notification dashboard React
```

```
### Fichiers cles
- `iot-security-backend/app/domains/intelligence/ml_service.py`
- `iot-security-backend/scripts/train_anomaly_detector.py`
- `iot-security-backend/scripts/train_iot_iforest.py`
- `iot-security-backend/scripts/update_pipeline_shap.py`

---

## 3.5 Explicabilite IA (SHAP)

### Probleme
Un modele ML qui detecte une anomalie sans explication n'est pas utilisable en securite industrielle. L'analyste doit savoir *pourquoi* une alarme est declenchee.

### Solution : SHAP TreeExplainer
Pour chaque prediction, SHAP calcule la contribution de chaque feature :
```

```
Exemple : Alerte sur capteur Weather
    -> Temperature = 44.4°C (contribution : +0.32 -- anormalement haute)
    -> Δt = +8.2°C (contribution : +0.45 -- saut brusque)
    -> Humidite = 12% (contribution : +0.18 -- anormalement basse)
    -> Heure = 14h (contribution : +0.02 -- neutre)
    -> Score final : -0.82 -> ANOMALIE
```

```
Les 3 features les plus contributrices sont exposees via l'API `/api/shap/latest` et affichees dans le panneau XAI du dashboard React (`XaiPanel.tsx`).

---

## 3.6 Concept Drift Monitoring

### Probleme
Les distributions des donnees IoT changent avec le temps (saisons, usure des capteurs). Un modele entraine sur des donnees de mars peut devenir inefficace en ete.

### Solution : Test KS-2-Sample
- **Reference** : Distribution des scores sur les donnees d'entrainement
- **Periode courante** : Distribution des 1000 dernieres predictions
- **Test** : Kolmogorov-Smirnov a 2 echantillons
- **Seuil** : p-value < 0.05 -> derive detectee -> necessite re-entrainement

Le statut de derive est expose via `/api/v1/drift/status` et l'historique via `/api/v1/drift/history`.

---

# Partie IV : API & Integration

## 4.1 Endpoints Principaux

### WebSocket
- `WS /ws/stream` -- Streaming temps reel de telemetrie + scores ML

### REST API
| Endpoint | Description |
|----------|-------------|
| `GET /api/telemetry` | Dernieres 150 mesures |
```

```
| `GET /api/devices` | Tous les devices |
| `GET /api/shap/latest` | Derniere analyse SHAP |
| `POST /api/attack/toggle` | Activer/desactiver simulation d'attaque |
| `GET /api/v1/devices/stats/summary` | Statistiques des devices |
| `POST /api/v1/devices/{id}/analyze` | Analyse ML d'un device |
| `GET /api/v1/alerts` | Alertes DPI (filtrables) |
| `GET /api/v1/alerts/summary` | Statistiques d'alertes |
| `GET /api/v1/drift/status` | Statut de la derive conceptuelle |

---

# Partie V : Bilan & Perspectives

## 5.1 Resultats

| Domaine | Livrable | Statut |
|---------|----------|--------|
| Base de donnees | SQLModel + DDD + Repository Pattern | Fait |
| MQTT DPI | 6 regles d'inspection temps reel | Fait |
| Simulateur | Streaming asynchrone 3 capteurs | Fait |
| EDA | Analyse exploratoire des 3 capteurs | Fait |
| Feature Engineering | 25 features temporelles + normalisation | Fait |
| Broker MQTT | Mosquitto integre au stack Docker | Fait |
| Client MQTT | gmqtt asynchrone dans FastAPI | Fait |
| Digital Twin | Synchronisation temps reel via MQTT | Fait |
| Isolation Forest | 4 modeles, 99%+ de detection | Fait |
| SHAP | Explicabilite des decisions IA | Fait |
| Concept Drift | Surveillance KS-test | Fait |
| Dashboard React | Interface temps reel (Vite + TS) | Fait |

## 5.2 Technologies

- **Backend** : Python 3.10+, FastAPI, SQLModel, SQLAlchemy 2.0, asyncpg
- **IA/ML** : scikit-learn (Isolation Forest), SHAP, scipy, joblib
- **Protocoles** : MQTT (gmqtt, Mosquitto)
- **Base de donnees** : PostgreSQL, SQLite (fallback)
- **Frontend** : React 19, TypeScript, Vite, Tailwind CSS, Recharts, Framer Motion
- **Streaming** : WebSockets (FastAPI)

## 5.3 Perspectives (Phase 3)

- Pipeline CVE (Common Vulnerabilities and Exposures) avec scoring predictif
- Interface Threat Intelligence enrichie
- Durcissement securite (TLS/mTLS, RBAC)
- Tests d'intrusion sur les scenarios d'attaque simules

---

*Document confidentiel — PeakSoft Security Platform — Juin 2026*
```